Software Engineering Midterm

V.Stiles
Dr. Tankeh |Bowie State University

1. Introduction

For this project, students were expected to create a C program that creates an office hour schedule for professors. The program is designed to read exactly two input files, one containing information about the professors and one containing information about the classes that they teach. The program then prints the professor's schedule along with their office hours, and a disclaimer that professors are not to be interrupted during class times. The program prints via a sample text document, a Word document, and an Excel document.

a. Requirements

Prior to writing the code, I listed the requirements based on whether they were functional or non-functional, as described in the course textbook [1].

Functional Requirements:

- Read exactly two input files only
- List each professor's class
- Assign number of necessary office hours to each professor
- Ensure office hours do not overlap with class time
- Print the results in a table style of choice

Non-Functional Requirements:

- Build and run in a Linux environment
- No hard-coding of any information

2. Program Build

The program was built modularly so that each code could be tested and refined independently of the proper functioning of another code. First I built the parser (parser.c), then I built the scheduler (scheduler.c). I coded the way that the program should handle output. I then created the Makefile and the static library, which is discussed later. Finally, added the python and prolog portions to the code. This follows the incremental development process model from Chapter 2 of the textbook, stating that each increment adds working functionality so problems are found early and the system can be shown at every stage instead of only at the end [1].

The program works in three steps. First, it reads the locally stored input files, then office hour are assigned, and then the output gets printed. First, the parser uses parse_professors() and parse_courses() to read each line with fgets(). The lines are split with a pipe (|) as a delimiter for all the information except for the course list, where commas are used as delimiters. Class times are in 12-hour standard time, so the code also converts these times to minutes from 12 am, so they can be easily compared to one another. The program also has a built-in check to ensure that every course listed in one of the input files also appears in the other file. I used local input files that are stored in the same program folder. To assign office hours, the scheduler.c file finds the days the professors teach, splits the hours only across the days they teach. It tries to start office hours right after one of the professors classes but if that time is not free, then it tries a slot every 15 minutes between 9 am and 5 pm. In the program, I ensured there is no overlap by defining overlap if an event starts before another ends. The program only places office hours where they do not overlap any other events. To

save the output and print it, the program prints a class schedule table and an office hours table for each professor followed by, “Instructors are not to be interrupted during class times.”, like in the sample output given to us. It saves the report to a text file called schedule_report.txt and saves the data to schedule_data.txt for the Python script.

As required, the program does not include any hard-coding. So, if the input files change, then the output files change.

3. Program Organization

a. Files and File Types

File Name	Purpose
parser.h/parser.c	Reads both input files, converts times, and ensures files point to the same information
scheduler.h/scheduler.c	Assigns office hours
schedule.h	Holds structs for professor, course, officeHour, ProfessorList, and courseList
output.h/output.c	Prints the tables and writes the data file
Makefile	Runs all files
make_schedule.py	Creates Word and Excel files
make_prolog.py	Writes Prolog facts and rules to schedule.pl
ask_prolog.py	Asks Prolog questions from Python using pyswip
queries.pl	Questions answered by Prolog

b. Static Library

A static library is a .a file that combines multiple compiled file objects into one [2,3]. The static library in this project was made from the parser.c file, scheduler.c file, and the output.c file to make libschedule.a. This demonstrates software reuse from Chapter 7 of the textbook, as it shows that another program could link this library and use the same functions without copying the code [1]. I used the $gcc -c command to compile them, and $ar rcs libschedule.a parser.o scheduler.o output.o, where the r,c, and s options added the files, created the archive, and built the index so the linker could quickly find the functions. To linked this library I used $gcc main.o -L. -lschedule -o schedule. This told the compiler what folder to look in for the library and what it was called. The compiler then pulls in the functions my main program needs from the library. For this to work, the main program (main.o) needed to be listed first so the compiler knows what is missing before it searches the library [4].

c. Makefile

A Makefile is a list of rules that tells the computer how to build a program and which files each part depends on [5]. When I type make, it compares the dates of the files. If a file has changed since the last build, Make rebuilds only the parts that depend on it and skips

everything else. For example, when I edited only the scheduler code, Make recompiled just that one file, rebuilt the library and relinked the program, without touching the other files. My Makefile also uses macros, which are short names for settings like the compiler, so a change only has to be made in one place. This kind of automatic system building is part of what Sommerville calls configuration management in Chapter 7 [1], and keeping my code on GitHub handles the version management side.

The Makefile also automates everything after compiling. With the make command, it builds the library and the program, runs the program on my two input files, uses Python to create the Word and Excel files, and runs the Prolog questions. The Python and Prolog lines start with a dash, which tells Make to keep going if that step fails. I added this because my original CoCalc project has no internet access, so those tools could not be installed there, but the C program still builds and runs. When I moved the project to Ubuntu on my device, these sections ran fully. I expand on Python and Prolog in the next section.

4. **Python and Prolog**

For the second part of the project, I used Python and Prolog to answer questions about the schedule. My Python script, make_prolog.py, reads the two input files and the office hours the C program created, and turns them into Prolog facts and rules. In Prolog, a fact is something that is true, while a rule tells Prolog how to derive answers from the facts. With my sample data, the script creates 31 facts and saves them, along with the rules, in a Prolog file. Prolog then stores this knowledge and answers questions about it. I ask the questions two ways: directly in Prolog, and from Python using the pyswip library, as demonstrated in the assignment. Each language used in this program serves a different purpose. C language does the scheduling, Python converts the data, and Prolog answers the questions. This follows the pipe-and-filter idea from Sommerville, Chapter 6, where each step passes its output to the next [1].

5. **Compilation and Testing**

I started the project in CoCalc, but CoCalc has no internet access, so I could not install Prolog or the Python libraries there. To run every part, I installed WSL (Linux inside Windows) on my own computer, added the C compiler, SWI-Prolog and the Python libraries, and downloaded my project. From there, one make command builds and runs everything. Working in two places like this is what Sommerville calls host-target development [1], and using a Makefile with standard Linux tools is what allowed the same code to run on both. On CoCalc, the C program still builds and runs; only the Python and Prolog steps are skipped.

I tested each part as I built it, following the development testing ideas in Chapter 8. Along with normal input, I tested unusual and invalid cases: running the program with only one file name, giving it a file that doesn't exist, adding blank lines and lines with missing fields, leaving out a professor's office room, and listing a course in one file that wasn't in the other. In each case the program printed a clear message or skipped the bad line instead of crashing. My first version crashed on missing fields, which is what led me to add those checks. I also confirmed that editing one file made Make rebuild only what depended on it, that the program still ran after deleting the library, and that Prolog answered all my questions correctly. After each fix, I ran the earlier tests again to make sure nothing else had broken.

6. **Sample Output**

```
stiles@LAPTOP-M2PF7JRO:~/Office-Hours-Midterm-Project$ make
./schedule faculty.txt classes.txt
Read 3 professors and 5 courses

================================================================================
Dr.Thomas - Computer Science
================================================================================
CLASS SCHEDULE
Course                              Times                                   Location
COSC208 (Discrete Structures)       Wednesday 5:00 PM to 7:30 PM            CS300
COSC214 (Data and Algorithms)       Monday & Wednesday 11:00 AM to 1:15 PM  CS330

OFFICE HOURS (4 hours per week)
Days                                Times                                   Location
Monday & Wednesday                  1:15 PM to 3:15 PM                      CS201

NOTE: Instructors are not to be interrupted during class times.
```

```
Q1: Who teaches COSC214?
   Dr.Thomas
Q2: Which office hours are on Wednesday?
   Dr.Jones: 10:45 AM to 11:45 AM
   Dr.Thomas: 1:15 PM to 3:15 PM
Q3: Which days does Dr.Smith teach?
   [thursday,tuesday]
Q4: Where does MATH220 meet?
   MATH100
```

Acknowledgements

I created a reference list as various sources helped in my understanding of this modular program. I referenced the textbook and sources explaining static libraries, linking libraries, Linux-based projects, and python. Grammarly intelligence tools were used to correct grammar, spelling, and fluidity mistakes only in the report. The Makefile uses -Wall command to allow the C compiler to warn about common coding problems that catch variable issues, missing return values, and typos. VS Code C and C++ extension was used during the very early coding steps, prior to transferring it to CoCalc. This extension shows potential coding issues as the code is being created.

References

[1] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2016.
[2] D. A. Wheeler, “Static libraries,” in Program Library HOWTO. The Linux Documentation Project. [Online]. Available: https://tldp.org/HOWTO/Program-Library-HOWTO/static-libraries.html. Accessed: Sept. 5 2026.
[3] Free Software Foundation, “ar,” in GNU Binary Utilities. [Online]. Available: https://sourceware.org/binutils/docs/binutils/ar.html. Accessed: Sept 8 2026.
[4] Free Software Foundation, “Options for linking,” in Using the GNU Compiler Collection (GCC). [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Link-Options.html. Accessed: Oct. 8 2026.
[5] R. M. Stallman, R. McGrath, and P. D. Smith, GNU Make: A Program for Directing Recompilation. Boston, MA, USA: Free Software Foundation. [Online]. Available: https://www.gnu.org/software/make/manual/make.html. Accessed: Sept. 5 2026.

Used as a learning tool, but not referenced in the report:

The Linux man-pages project, “strtok(3) — Linux manual page.” [Online]. Available: https://man7.org/linux/man-pages/man3/strtok_r.3.html. Accessed: Sept. 5 2026.

P. Blackburn, J. Bos, and K. Striegnitz, Learn Prolog Now! London, U.K.: College Publications, 2006. [Online]. Available: https://lpn.swi-prolog.org/. Accessed: Oct. 10 2026.

PySwip contributors, “Get started,” PySwip Documentation. [Online]. Available: https://pyswip.readthedocs.io/en/latest/get_started.html. Accessed: Oct. 9 2026.

S. Canny, python-docx Documentation. [Online]. Available: https://python-docx.readthedocs.io/. Accessed: Oct. 9 2026.